Neural networks and deep learning, without the maths wall

What is a neural network, and when is one the right tool?

AI and Data Science Professional | Session 20 of 24 | Module E: Neural networks and text
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Describe an artificial neuron, layers, a loss function and gradient descent in plain words, with the real terms

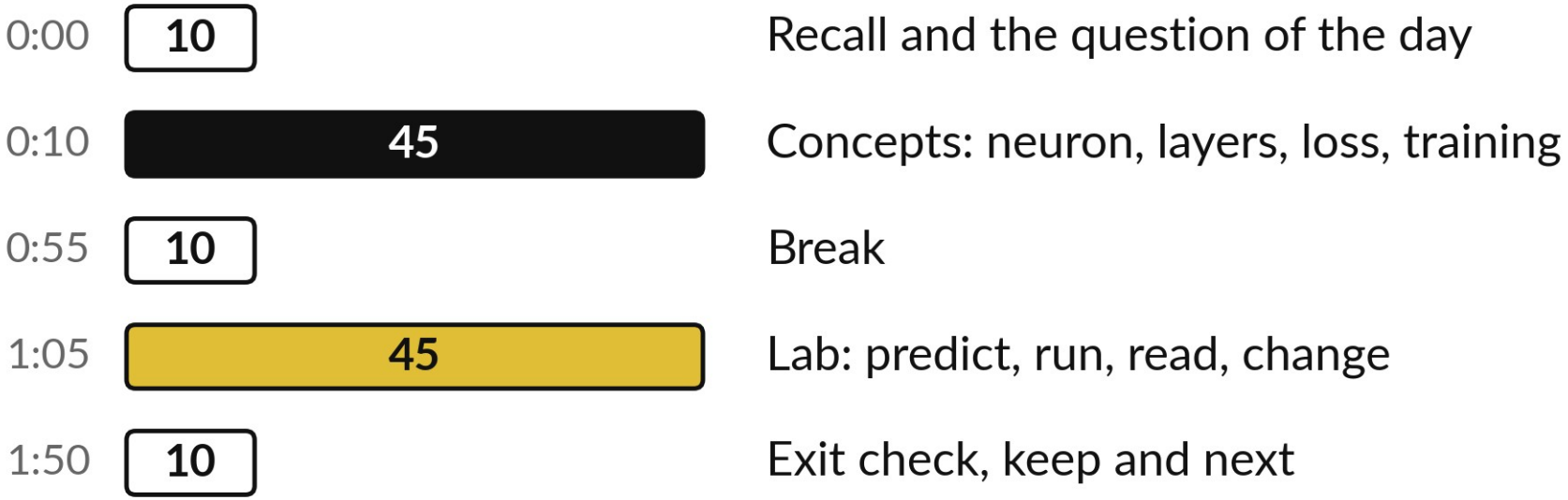


Run a neuron and a real gradient descent loop in numpy, and watch a network overfit



Compare an MLPClassifier with a random forest on the loans data and say when deep learning is worth it

Your two hours: network ideas first, then the neural network lab



Ideas with pictures first. Then you open the notebook and TensorFlow Playground.

Three questions from last time

1. Why do we split a time series by date?

2. What must a forecast beat before you trust it?

3. Why do we group by district before we shift a lag feature?



Answer aloud first. Then we check together.

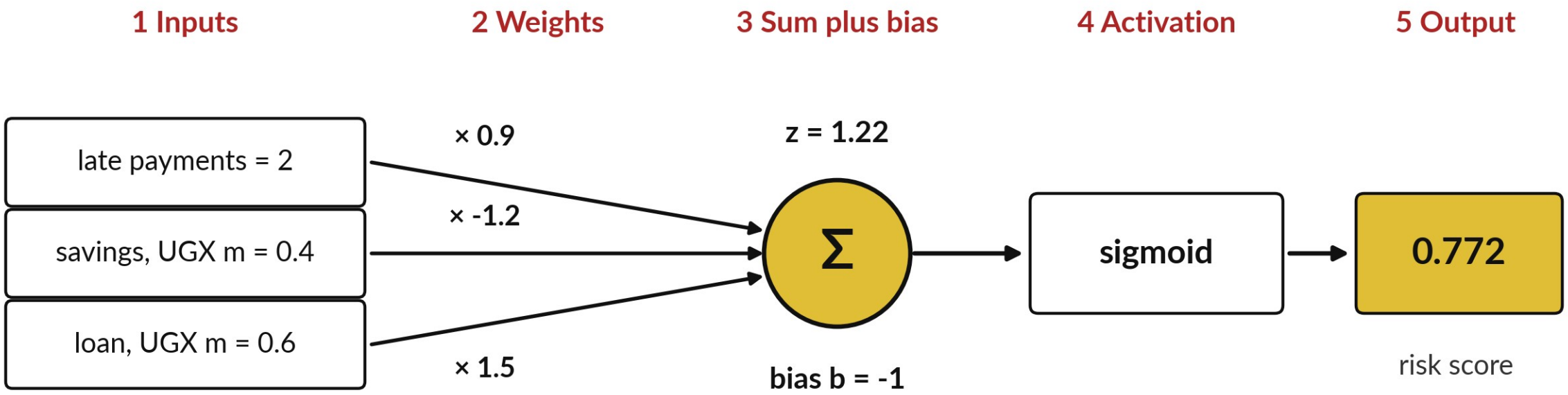
Answers to the three questions from Session 19

Question	Short answer
Why do we split a time series by date?	A random split lets the model see months after the test months, which is data leakage. Next month always lies after the data you have.
What must a forecast beat before you trust it?	The baseline model: the seasonal naive forecast that copies the same month from last year.
Why do we group by district before we shift a lag feature?	Without groupby, the first month of a district borrows its lag from the district above it in the file.

Words we use today

	Word	What it means
	**Artificial neuron (perceptron)**	A small calculation: a weighted sum of the inputs plus a bias, passed through an activation function.
	**Activation function (sigmoid, ReLU)**	The bend applied to the weighted sum. Sigmoid gives a score from 0 to 1. ReLU gives 0 below zero and the total above it.
	**Feedforward neural network (multilayer perceptron, MLP)**	Neurons arranged in an input layer, hidden layers and an output layer, with values moving forward only.
	**Gradient descent and backpropagation**	Training: repeated small steps that lower the loss function. Backpropagation works out how much each weight changed the loss (its gradient).
	**Transfer learning (pretrained model)**	Start from a network someone else trained and train only a new last layer on your own data.

An artificial neuron is a weighted sum plus a bias, passed through an activation function



Illustrative loan and weights, never trained. Spreadsheet word for the sum: SUMPRODUCT.

You already know the parts: the Excel word and the industry term

	In Excel	Industry term	Plain meaning
	**SUMPRODUCT of inputs and multipliers**	Weighted sum	Multiply each input by its weight, then add
	**The multiplier beside each input**	Weight (parameter)	How much the input counts; training adjusts it
	**The constant added at the end**	Bias (intercept)	Where the neuron starts
	**IF or MAX around a formula**	Activation function	Bends the total: sigmoid or ReLU
	**Helper columns before the answer**	Hidden layer	Neurons that build new variables
	**Total of an error column**	Loss function	One number for how wrong we are

The neuron as formulas: z is the weighted sum plus bias, and a is the activation

Weighted sum plus bias

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

Sigmoid activation

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

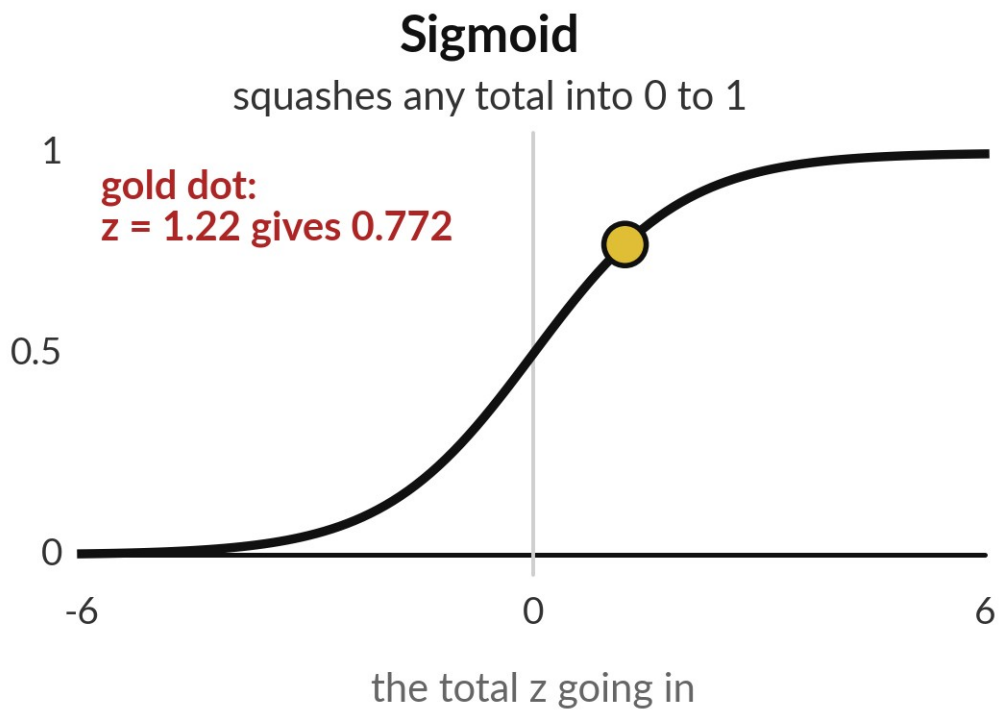
Our neuron

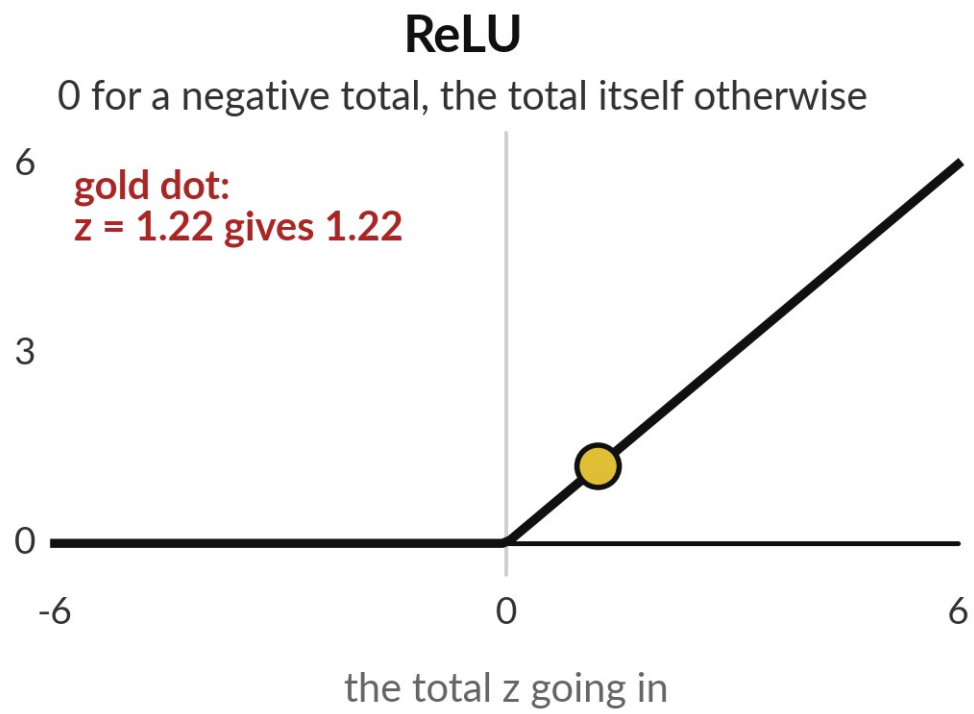
$$z = 0.9 \times 2 + (-1.2) \times 0.4 + 1.5 \times 0.6 + (-1) = 1.22$$

Symbol	What it means	In our data
x_i	An input: one variable of the loan.	x = 2, 0.4, 0.6
w_i	A weight: how much that input counts.	w = 0.9, -1.2, 1.5
b	The bias (intercept).	b = -1
z	The weighted sum plus the bias.	z = 1.22
σ(z)	The sigmoid: squashes z into 0 to 1.	σ(1.22) = 0.772

This is the logistic regression you met in Session 15, packaged as one unit.

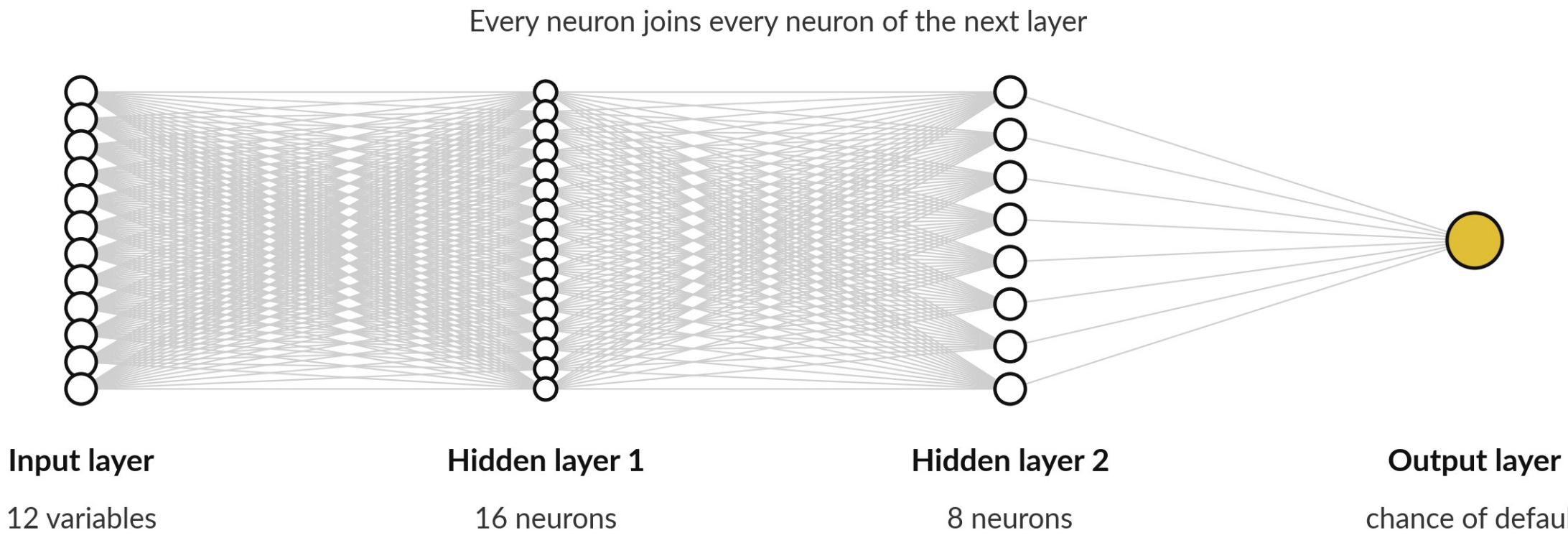
Activation functions bend the total: sigmoid gives a probability and ReLU is common in hidden layers





ReLU (rectified linear unit): 0 for a negative total, the total itself otherwise.

Layers stack neurons into a feedforward neural network (multilayer perceptron, MLP)



Our loans network: 12 variables in, 16 and 8 neurons, 1 probability out. 353 weights and biases.

Reading the network: layers, parameters and the arithmetic behind the count

	Term	Plain meaning	In our loans network
	**Input layer**	One node per variable. It holds the data.	12 variables
	**Hidden layer**	Neurons that build new variables from the old ones.	16 neurons, then 8
	**Output layer**	The prediction.	1 sigmoid neuron: chance of default
	**Parameters**	Every weight and bias that training adjusts.	12×16+16, 16×8+8, 8×1+1 = 353
	**Feedforward**	Values move from input to output, never backwards.	when predicting

More neurons mean more parameters, and more room to memorise the training set.

The loss function is one number for how wrong the predictions are

Squared error loss for one record

$$L = (\hat{y} - y)^2$$

Our one-weight example: x = 2 and the target is 6

$$\hat{y} = 2w \quad L(w) = (2w - 6)^2$$

At w = 0

$$L(0) = (0 - 6)^2 = 36$$

Symbol	What it means	In our data
y	The actual value (the target).	y = 6
ŷ (y-hat)	The prediction of the model.	ŷ = w × x = 2w
w	The weight that training adjusts.	starts at w = 0
L	The loss for one record.	L(0) = 36

A classifier usually uses log loss (cross-entropy). Training means lowering the loss.

Gradient descent walks the weight downhill: w goes from 0 to 2.95 in 8 steps



Gradient descent lowers the loss one small step at a time.

Gradient: the slope of the loss at the current weight.

Learning rate: the size of each step.

New weight = old weight minus learning rate times gradient.

Read the recipe: gradient descent on one weight

```
x, target, w, lr = 2.0, 6.0, 0.0, 0.05
for step in range(1, 9):
    loss = (w * x - target) ** 2
    grad = 2 * (w * x - target) * x
    w = w - lr * grad
    if step in (1, 2, 4, 8):
        print(step, round(w, 3), round(loss, 3))
```

```
1 1.2 36.0
2 1.92 12.96
4 2.611 1.68
8 2.95 0.028
```

In plain words

Line 1 sets the input, target, start weight and learning rate.

Line 3 is the loss. Line 4 is the gradient.

Line 5 moves the weight against the slope.

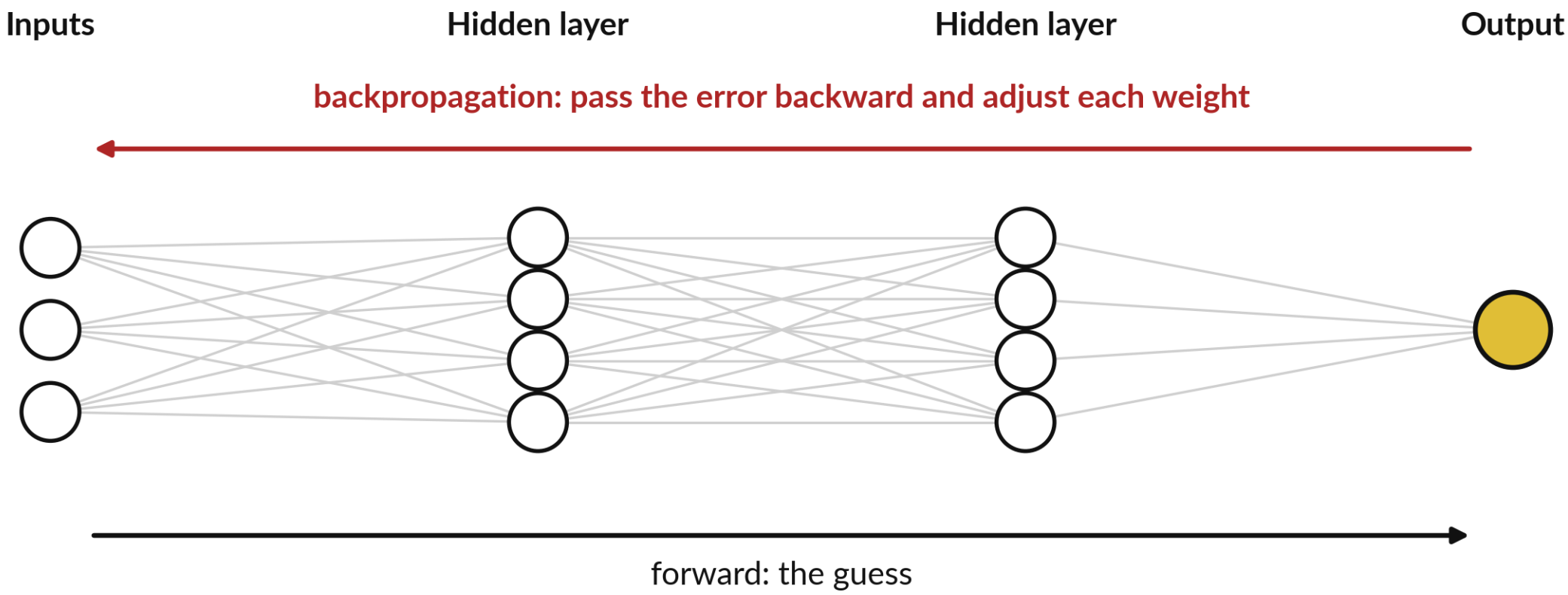
Real output: w climbs to 2.95, the loss falls toward 0.

The learning rate decides whether training settles, bounces or blows up

	Learning rate	Weight after 8 steps	Loss after the 8th update	What happens
✓	**0.05**	2.95	0.010	Settles: each step is smaller than the last.
?	**0.25**	0.0	36.0	Bounces: the weight jumps between 0 and 6 and never settles.
✗	**0.5**	-19,680	1,549,681,956	Blows up: each step overshoots by more than the last.

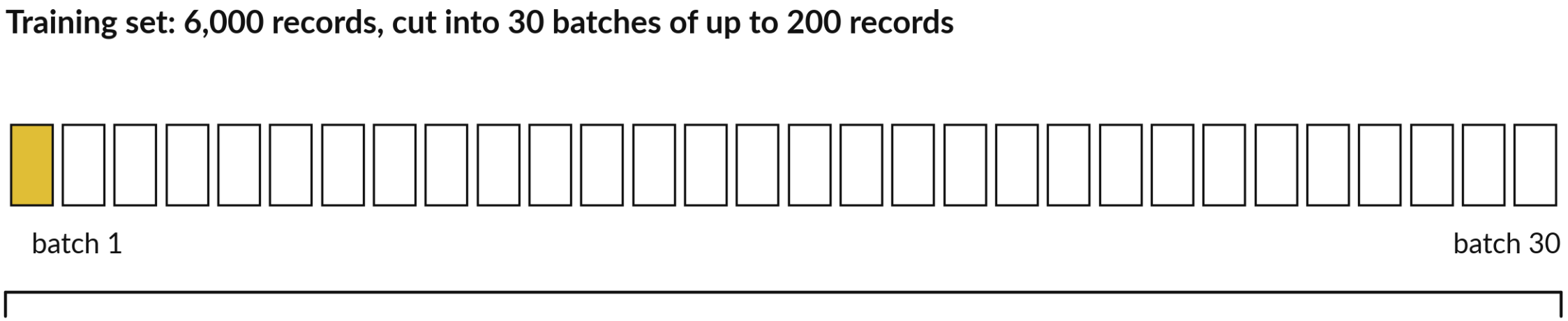
Same loop, same target. Best weight: 3, loss 0. The recipe on the last slide prints the loss before each update, so its last line shows a larger loss.

Backpropagation works out how much each weight changed the loss, so gradient descent can adjust them all



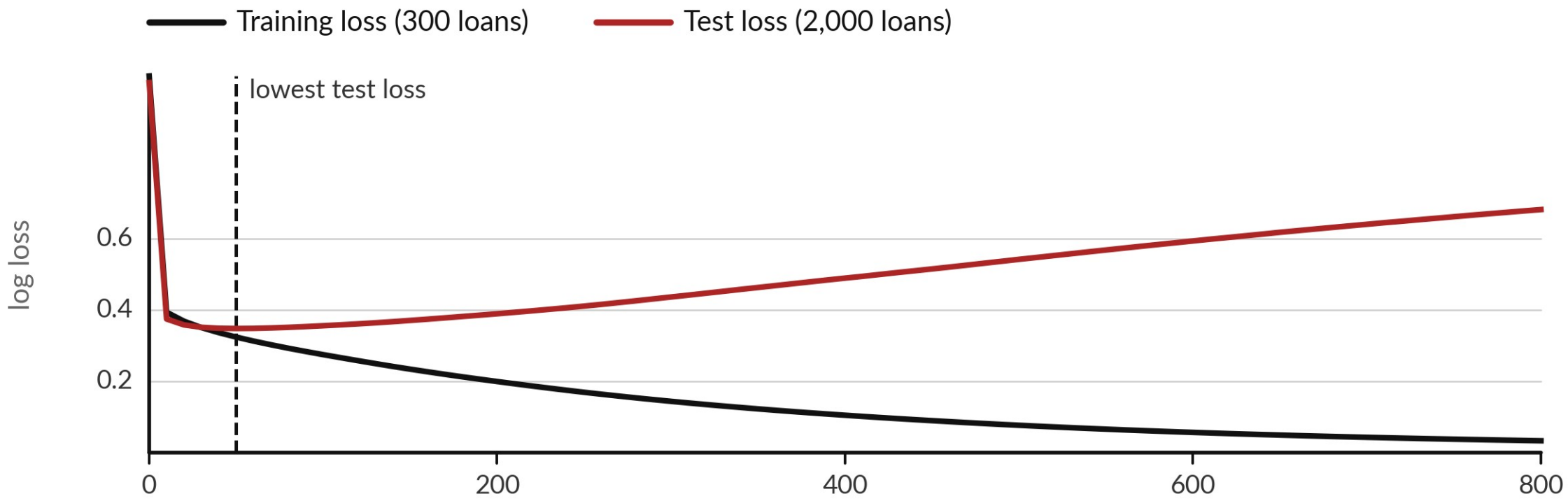
Backpropagation: send the error backwards to find each weight's gradient, its effect on the loss.

One epoch is one pass over the training set, made of many batches



MLPClassifier uses batches of up to 200 records by default (check your version).

Overfitting again: after epoch 50 the training loss keeps falling and the test loss rises



A 12-64-64-1 network on only 300 training loans. Epochs run along the bottom.

Four ways to fight overfitting in a network



Use fewer neurons or fewer layers: a smaller network has less room to memorise.



Regularisation: MLPClassifier has an alpha setting that penalises large weights.



Early stopping: stop training when the validation loss stops improving (early_stopping=True).



More training records: the most reliable cure when you can get them. Always judge on the test set.

Network or forest on a table? Make the network earn its place

Neural network (MLP)

Needs imputing and scaling first.

Many settings. Trains with Adam by default, a smarter gradient descent.

Hard to explain to a manager.

Random forest

Works on raw numbers, no scaling.

Few settings, strong defaults.

Shows which variables mattered.

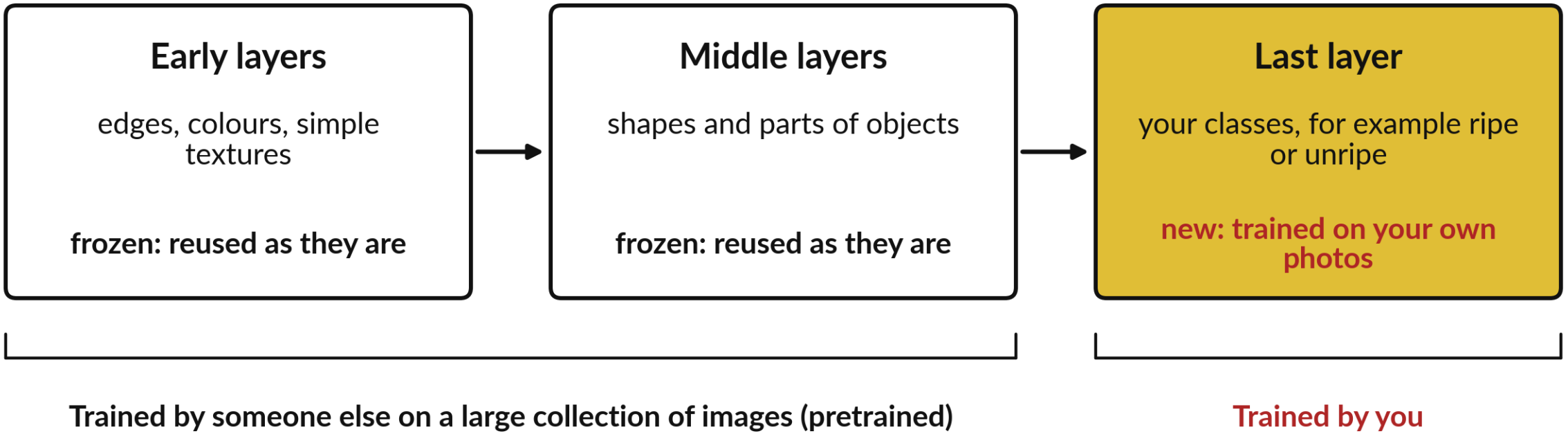
On a table, start simple. Keep the network only if it wins by enough to matter.

On the loans table a numpy network does no better than logistic regression: 353 parameters, no gain

	Model	Parameters	Test AUC	Where the number comes from
	**Logistic regression (numpy)**	13	0.758	Computed here in numpy
	**Network 12-16-8-1 (numpy)**	353	0.742	Computed here in numpy
	**MLPClassifier (scikit-learn)**	353	You compute it	Your lab
	**RandomForestClassifier**	many trees	You compute it	Your lab

Same 8,000 loans and the same 2,000 test loans as Sessions 14 to 17 (251 defaults). collection_calls is left out because it leaks the answer: with it the AUC would be above 0.99.

Transfer learning reuses a pretrained model and trains only a new last layer



Pretrained model: trained by someone else. Inference: using a trained model to predict.

Four terms for using a network somebody else trained

	Term	Plain meaning	Example
	**Pretrained model**	A network already trained on a large dataset by someone else.	An image classifier published on the Hugging Face Hub
	**Inference**	Using a trained model to predict. No training happens.	Classify one new photo
	**Transfer learning**	Reuse the early layers and train a new last layer on your own data.	A few hundred labelled photos of your own crops (illustrative)
	**Fine-tuning**	Also adjust some of the pretrained weights, with a small learning rate.	Needs more data and more care

Before you use a pretrained model at work, read its model page: the licence, the training data and the download size.

When deep learning is worth it: match the tool to the kind of data

Kind of data	Usually the best first tool	Why
A table of numbers: loans, orders, clinic visits	Logistic regression, random forest, gradient boosting	Few records per variable, and you must explain the result
Images and photos	Pretrained network with transfer learning	Pixels are not meaningful variables on their own
Text and speech	Pretrained language or speech model	Meaning depends on the words around each word
Very large data with a hard pattern	Deep learning trained on a GPU	Needs a lot of data and computing power

This is my understanding of the usual pattern. Always test the simple model first on your own data.

Where you will use this at work: judge the network, reuse the pretrained model

Your job	The task you meet	What today lets you do
Credit analyst at a bank, using scikit-learn	Default or late repayment on a table of loans	Compare a network, a forest and logistic regression on one split, then recommend the simplest that is good enough.
Agriculture extension officer with a phone	Photos of crop leaves (illustrative)	Know that a pretrained image model with transfer learning is the starting point, and what labelled photos it needs.
Data team lead reviewing a vendor pitch	A vendor says its AI forecast uses deep learning	Ask which baseline it beats, on which test set, and by how much.
Analyst handing results to managers	A score of 99 percent on training data	Recognise overfitting and ask for the test score.

Read the recipe before the lab: a network in a scikit-learn Pipeline

```
mlp = make_pipeline(
    SimpleImputer(strategy='median'),
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(16, 8),
                  max_iter=500, random_state=42))
mlp.fit(X_train, y_train)
y_prob = mlp.predict_proba(X_test)[:, 1]
print(roc_auc_score(y_test, y_prob))
```

In plain words

make_pipeline chains the imputer, the scaler and the network in order.

hidden_layer_sizes=(16, 8) means two hidden layers.

max_iter=500 limits the epochs. random_state=42 makes the run repeatable.

predict_proba gives the chance of default. roc_auc_score scores it on the test set.

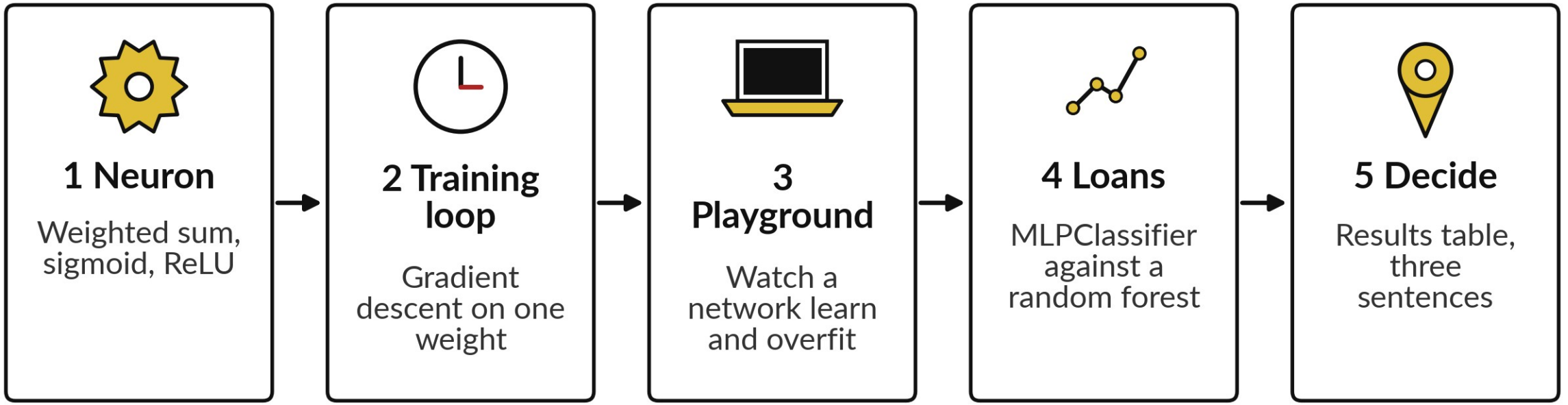
You run this in the lab, and the AUC appears in your notebook.



Part 2: the neural network lab

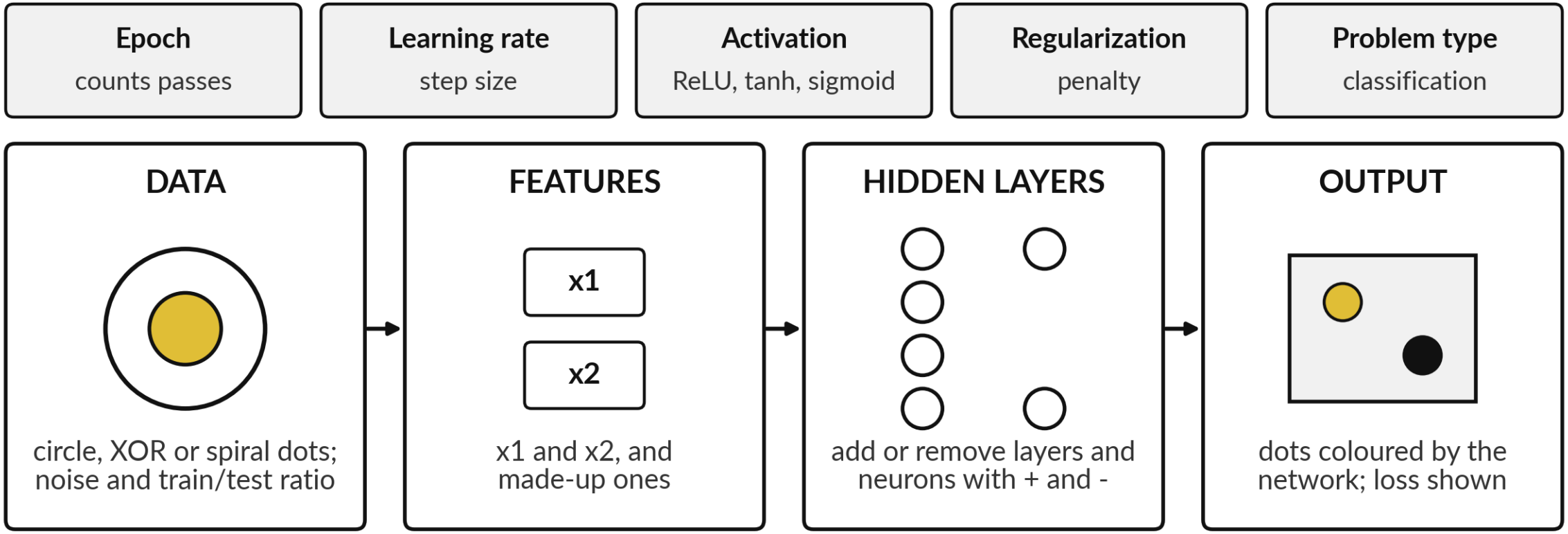
Close the slides. Open the notebook.

The lab has five parts and ends with a recommendation to the credit team



Predict, run, read, change one thing. Automatic checks tell you when a number is right.

TensorFlow Playground lets you watch a small network learn in your browser



Sketch of the screen. The real page may look different. Free, in the browser, no account.

Four Playground challenges: change one thing and write down what you see

	Challenge	What to change	What to record
	**1 Circle**	Start with no hidden layer. Then add one hidden layer with 4 neurons.	Can it separate the circle each time? How long does it take?
	**2 Spiral**	Add a second hidden layer and more neurons.	The test loss before and after. Which size is enough?
	**3 Overfitting**	Raise the noise, use few training points, train for a long time.	When do the training loss and the test loss separate?
	**4 Learning rate**	Try a very small and a very large learning rate. Try ReLU and tanh (a bend that gives values from -1 to 1).	Which settings fail, which are slow, which are fast?

Ten minutes. Write in the notebook text cell. There is no single right answer: record what you see.

Optional: use a pretrained image classifier in Colab

Only if you finish early.



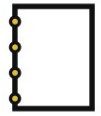
The first run downloads the model weights from the Hugging Face Hub (free, tens of megabytes or more).



Read the model page first: licence and size.



Never upload photos of people or customers.



Needs torch or TensorFlow: use the Colab CPU runtime.

Check your numbers before you move on

	What you compute	You should get
	Neuron score for the loan (1, 0.8, 0.3), and with first weight 4.5	0.352 and 0.952
	ReLU of the weighted sum	about 0.0 for z = -0.61
	Weight after 8 steps at learning rate 0.05	2.95
	Loss after 8 steps at learning rate 0.5	above 100 (it blows up)
	Parameters of a 10-16-8-1 network, batches per epoch	353 and 30
	Loans kept, training records, test records (defaults)	8,000, 6,000 and 2,000 (251)
	Test AUC of the MLPClassifier and of the forest	between 0.70 and 0.78; yours will differ
	Training AUC minus test AUC of the big network on 300 loans	above 0.1

If your number differs, find the task that went wrong.

Five likely mistakes in the lab and how to fix them

What you see	What it means	What to do
ConvergenceWarning: maximum iterations reached	Training stopped before the loss settled.	Raise max_iter, or accept it if the AUC is stable.
ValueError: input contains NaN	Blank incomes reached the network.	Put SimpleImputer first in the Pipeline.
AUC above 0.99	collection_calls is in X. It leaks the answer.	Drop it from the variables.
The loop weight becomes huge or nan	The learning rate is too big.	Use 0.05 and look at each step.
Your AUC differs from the slides	Random starting weights, or a different split.	Up to about 0.03 is expected. Above 0.05 means a different split or variable list.

Exit check: three questions, then compare with the short answers

Question	Short answer
What does one artificial neuron compute?	A weighted sum of its inputs plus a bias, passed through an activation function such as sigmoid or ReLU.
In gradient descent, what does the learning rate control, and what happens if it is too big?	The size of each step. Too big and the weight overshoots, bounces or blows up.
A network scores 1.00 AUC on the training loans and 0.65 on the test loans. What is wrong and what do you try?	Overfitting. Try a smaller network, regularisation, early stopping or more records, and quote the test score.

Save your notebook and your recommendation

What you keep from today

Keep your lab notebook with its results table and your three-sentence recommendation: network, random forest or logistic regression for the credit team, and why.

Our question today

What is a neural network, and when is one the right tool?

Next session

Session 21: Text as data. We turn customer messages into numbers with bag of words, TF-IDF and embeddings.

Take away

Your lab file

A network notebook

The key test

Does it beat the simple model?

Next session

Text as data